SO - FileSystem

Cos'è un File e un FileSystem

Un File System è un modo per organizzare e memorizzare delle informazioni. Queste informazioni sono contenute su un disco e possono essere lette. Chiamiamo tutto ciò File. Questo è un metodo di astrazione che permette di nascondere all'utente i dettagli di come e dove le informazioni sono memorizzate e di come funzioni effettivamente il disco.
Questi file vengono identificati tramite:

- nome - che lo identifica
- un'estensione - indica una caratteristica specifica di un file
- degli attributi - ovvero un insieme di caratteristiche o metadati che forniscono ulteriori informazioni su un file

Quando un utente fa doppio clicl sul nome di un file, il programma assegnato alla sua estensione viene lanciato con il file come parametro.
ES: Se fai doppio clicl su un file.docx si avvia Microsoft Word ma contemporaneamente si avvia anche il file.docx

I file possono essere strutturati in tanti modi diversi:

- Sequenza Non Strutturata di Byte (sequenza di 0 e 1 qualsiasi in base al numero di byte - 1 byte = 8 bit)
- Sequenza di Record di lunghezza fissa
- File come Albero di Record con lunghezze variabili e un campo chiave in posizione fissa
- File eseguibili
- File di Archivio, ovvero raccolta di procedure di libreria (moduli) compilate ma non collegate

Accesso ai File

Nei primi Sistemi Operativi, per accedere ad un file stava solo il metodo sequenziale.

Con l'avvento dei dischi è stato introdotto l'accesso causale. È necessario per accedere rapidamente a record specifici.
Per specificare dove cominciare a leggere possono essere usati due metodi:

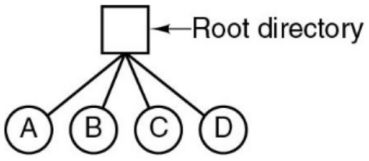
- Nel primo, ogni operazione read fornisce la posizione del file dalla quale iniziare a leggere.
- Nel secondo è fornita un'operazione speciale, seek, per impostare la posizione corrente.

Quali operazioni si possono effettuare sui File?

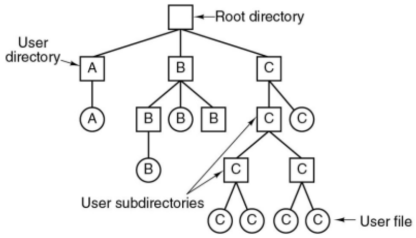
- **Create:** Creazione di un file senza dati
- **Delete:** Eliminazione di un file per liberare spazio sul disco, attraverso una specifica chiamata di sistema
- **Open:** Apertura di un file per consentire al sistema di caricare in memoria gli attributi e gli indirizzi del disco
- **Close:** Chiusura del file alla termine degli accessi per liberare spazio nelle tabelle interne
- **Read:** Lettura dei dati da un file, generalmente dalla posizione corrente, specificando la quantità di dati richiesti e fornendo un buffer per la loro memorizzazione
- **Write:** Scrittura di dati nel file, tipicamente alla posizione corrente, può comportare l'ampliamento del file o la sovrascrittura dei dati esistenti
- **Append:** Aggiunta dei dati solo alla fine del file, usata in alcuni sistemi operativi come forma limitata di scrittura
- **Seek:** Riposizionamento del puntatore del file su una posizione specifica per file ad accesso casuale, permettendo la lettura o la scrittura da quella posizione
- **GetAttributes:** Lettura degli attributi del file
- **SetAttributes:** Modifica degli attributi di un file da parte dell'utente, come la modalità di protezione o altri flag, dopo la creazione del file
- **Rename:** Ridenominazione di un file, utilizzata come alternativa al processo di copia ed eliminazione del file originale, specialmente utile per file di grandi dimensioni.

Cosa sono le Directory?

Le directory sono cartelle, anch'esse file che si torvano all'interno del file system e contengono file.
La directory più semplice è la **directory principale** (*root directory*).



Esistono sistemi a **directory gerarchici**. Queste rappresentano un'evoluzione fondamentale nell'organizzazione dei dati poiché permettono di strutturare le informazioni in una configurazione ramificata ad albero partendo da una radice comune chiamata root che si dirama in cartelle e sottocartelle.
Tutto ciò, rende possibile la gestione di ambienti multi-utente in cui ogni individuo possiede uno spazio privato isolato dagli altri utenti.
Oggi è lo standard universale per ogni sistema operativo moderno grazie alla sua flessibilità infinita nel creare nuovi rami per ogni progetto o necessità specifica dell'utente.



Come si definisce la posizione di un file?

La posizione di un file può essere definita secondo 2 percorsi:

- **Percorso Assoluto:** Inizia sempre dalla "directory principale" (*root*), che in sistemi tipo UNIX è rappresentata dal simbolo /
  Poiché traccia l'intero cammino dalla radice fino al file, quel percorso non indicherà dove si "si trova" l'utente in quel momento nel sistema.

- **Percorso Relativo:** Indica la posizione di un file o di una directory partendo esclusivamente dalla cartella in cui ci si trova in quel momento, anziché dalla radice del sistema.

Quali sono le operazioni sulle Directory?

- **create:** Creazione di una directory vuota con le voci '.' e '..'
- **delete:** Eliminazione di una directory, possibile solo se la directory è vuota
- **opendir:** Apertura di una directory per la lettura del suo contenuto
- **closedir:** Chiusura di una directory dopo la lettura per liberare risorse
- **readdir:** Restituisce la prossima voce in una directory aperta senza esporre la struttura interna
- **rename:** Rinomina una cartella, simile al rinomino di un file
- **link:** Crea un hard link, collegando un file esistente a un nuovo percorso condividendo l'i-node(blocco di dati presente sul disco)
- **unlink:** Rimuove una voce di una directory, cancellando il file se è l'unico link

Implementazione del File System

Il file system è un metodo usato per l'organizzazione e la conservazione dei dati su memorie non volatili.
I meccanismi di avvio del computer che permettono al sistema operativo di individuare e caricare il file system.
Studiamo 2 meccannismi di avvio:

- *Il Master Boot Record (MBR)*: è il settore (cella) di avvio principale che risiede all'inizio del disco e contiene le istruzioni fondamentali per il computer poiché ospita la tabella delle partizioni (ovvero lo schema che indica dove iniziano e finiscono le diverse sezioni del disco), e il codice necessario a individuare la partizione attiva per caricare il sistema operativo. Però è una tecnologia datata, che presenta dei limiti strutturali che non gli permettono di gestire dischi più grandi di 2 TB (2000 GB) o di avere più di quattro partizioni primarie, motivo per cui oggi viene sostituito dal sistema UEFI.

- *UEFI (Unified Extensible Firmware Interface)*: rappresenta lo standard di avvio moderno che sostituisce il vecchio BIOS e l'MBR, configurandosi come un vero e proprio mini-sistema operativo integrato nel firmware della scheda madre che permette al computer di interfacciarsi con l'hardware in modo molto più veloce e flessibile. Questo meccanismo non si limita a leggere il primo settore del disco ma utilizza la *GPT (GUID Partition Table)*, una struttura che risiede nei primi blocchi e che consente di gestire dischi di dimensioni immense fino a 8 ZiB e un numero teoricamente infinito di partizioni, garantendo inoltre un'elevata sicurezza grazie alla capacità di conservare una copia di backup della tabella delle partizioni nell'ultimo blocco del disco per prevenire la perdita di dati in caso di errori o corruzioni.

Una corretta implementazione dei file è fondamentale per garantire che i dati rimangano corretti e non si corrompano, che il sistema riesca a trovarli e caricarli rapidamente e che ogni porzione del disco venga utilizzata senza sprechi di spazio.
Esistono vari metodi di allocazione:

- **Allocazione Continua:** memorizzare ciascun file come una sequenza contigua (adiacente) di blocchi sul disco.
  - VANTAGGI: Semplice da implementare e Prestazioni di lettura eccellenti.
  - SVANTAGGI: Frammentazione del disco, Difficoltà nell'aggiungere nuovi file in un disco frammentato, Necessità di compattare il disco, operazione costosa.

- **Allocazione a Liste Concatenate:** configurare ciascuno come una lista concatenata di blocchi sul disco. Ovvero la prima parte del blocco è usata come puntatore al successivo, la seconda parte per i dati

  - VANTAGGI: Utilizza in modo efficiente tutti i blocchi del disco, Minimizza la frammentazione esterna.
  - SVANTAGGI: Accesso casuale estremamente lento, Dimensioni dei blocchi non più una potenza di due, riducendo l'efficienza

- **Allocazione a Liste Concatenate con FAT:** funziona come l'allocazione a liste concatenate, ma con la particolarità che vengono spostati i puntatori in una tabella di memoria . Ogni blocco del disco è rappresentato come una voce nella FAT, in memoria RAM.
  - VANTAGGI: Accesso casuale semplificato, poichè i puntatori sono in RAM.
  - SVANTAGGI: La tabella deve rimanere sempre in memoria, Occupa considerevole quantità di RAM, specialmente su dischi grandi.

Cosa sono gli i-node?

Gli *i-node* o *index-node* sono una struttura dati che elenca gli attributi (esclusi nome e contenuto) come permessi, proprietario, timestamp e gli indirizzi dei blocchi dei file. Ogni file e directory è rappresentato da un I-node univoco, indicizzato in una tabella di I-node.

Implementazioni delle Directory

Il ruolo della Directory è quello di collegare il nome testuale del file (leggibile dall'utente in formato ASCII) alle coordinate tecniche necessarie al sistema operativo per recuperare i dati fisici dal supporto di memoria.
A seconda del sistema, questa informazione può essere

- l'indirizzo del disco dell'intero file (allocazione contigua)
- il numero del primo blocco (linked list)
- il numero dell'i-node

I moderni file-system supportano nomi di file variabili da 1 a 255 caratteri.
Per fare ciò, le directory si possono strutturare in 2 modi:

- la prima struttura prevede che ogni voce inizi con un *header di lunghezza fissa* ogni file inizia con un'intestazione di dimensione fissa seguita dal nome del file. Per velocizzare la lettura da parte del computer, il nome viene "allungato" con dei caratteri di riempimento fino a raggiungere un punto preciso della memoria (l'allineamento).
  Quando un file viene cancellato, lascia uno spazio vuoto di dimensioni prestabilite: se il nuovo file che vuoi inserire ha un nome più lungo del precedente, non riuscirà a occupare quel posto. Si creano così dei "buchi" nella memoria che rimangono inutilizzati, causando uno spreco di spazio chiamato frammentazione.
- La seconda struttura prevede l'uso di un *Heap*. Ovvero utilizzare un "heap", in modo tale che, i nomi vengono scritti uno dopo l'altro senza dover rispettare confini precisi o aggiungere spazi vuoti di riempimento.
  Il grande vantaggio è la flessibilità: quando cancelli un file, lo spazio che si libera può essere riutilizzato facilmente da qualsiasi altro nome, indipendentemente dalla sua lunghezza. Non ci sono più "posti" troppo piccoli, perché i nomi occupano solo lo spazio di cui hanno strettamente bisogno.
  In passato, per trovare un file il computer doveva leggere tutta la lista dall'inizio alla fine, un processo molto lento se i file erano migliaia. Per risolvere il problema si è passati alle Hash Table, come avere un indice analitico che trasforma il nome del file in un numero (un codice). Questo numero ti dice subito in quale "cella" guardare, evitando di scorrere l'intera directory. Se più file finiscono nella stessa cella, vengono semplicemente collegati tra loro in una piccola lista

Come possono essere i file condivisi?

I file condivisi posso essere:

- Hard Link: Puntano direttamente all'i-node di un file condiviso. Il file non verrà rimosso fino all'eliminazione dell'ultimo link.

- Soft Link: Puntano al nome di un file piuttosto che all'i-node.
  Un soft link, non contiene i dati del file, ma solo il suo percorso (l'indirizzo di memoria). Se il file originale viene rimosso, il soft link diventa invalido perché continua a puntare a un percorso che ora è vuoto.

Entrambe le modalità hanno il seguente problema: i file con più percorsi possono essere processati più volte da programmi di backup o di ricerca

Gestione dello spazio sul Disco

Il disco rigido non salva i file in un unico ammasso disordinato, ma divide lo spazio in tante piccole "scatole" chiamate blocchi. La dimensione standard di 4KB per ogni blocco è stata scelta come un compromesso ideale.
Se i blocchi fossero troppo grandi, sprecheremmo spazio.
Se fossero troppo piccoli, il computer diventerebbe lentissimo.
I 4KB sono quindi il punto di equilibrio tra velocità di lettura e risparmio di spazio.

Cosa e quali sono i Blocchi Liberi?

Per salvare nuovi file, il sistema deve sapere quali blocchi sono vuoti. Esistono due modi principali per farlo:

- Linked List: Si utilizza una lista concatenata. In questa struttura, i blocchi liberi vengono collegati tra loro: ogni blocco vuoto contiene l'indirizzo del blocco libero successivo.
  Per ottimizzare le prestazioni e non sovraccaricare la RAM, il sistema operativo mantiene in memoria solo un "blocco di puntatori" alla volta; quando questo si esaurisce, viene letto un nuovo blocco dal disco.

- Bitmap: Un disco con n blocchi richiede una bitmap con n bit.
  I blocchi liberi sono indicati dal valore 1 nella mappa, quelli allocati dallo 0. È più efficiente rispetto alla linked list, tranne in dischi quasi pieni.

Esistono delle quote fisse per il disco. L'amministratore di sistema assegna a ciascun utente un numero massimo di file e blocchi, e il sistema operativo si accerta che gli utenti non superino la loro quota. Ogni apertura di file coinvolge il controllo degli attributi e degli indirizzi sul disco. Gli attributi includono l'identificatore del proprietario del file. In una tabella dei file aperti, viene contabilizzata la quota di ciascun utente. Ci sono 2 limiti, soft e hard. Il limite soft può essere temporaneamente superato, mentre il limite hard, no.

Il Gap di Prestazioni

Il problema centrale nella progettazione di un File System moderno risiede nella disparità di velocità tra i diversi livelli della gerarchia di memoria. Come analizzato, la **RAM** (memoria volatile) presenta tempi di accesso nell'ordine dei $10\,ns$, mentre i **dischi magnetici** (memoria non volatile di massa) operano nell'ordine dei $10\,ms$. Tale differenza può essere espressa dal rapporto:

$$Gap = \frac{T_{disk}}{T_{ram}} = \frac{10^{-2}\,s}{10^{-8}\,s} = 10^6 \tag{1}$$

Pertanto, bisogna progettare un file system con diverse ottimizzazioni per migliorare le prestazioni. L'introduzione degli **SSD** ha ridotto questo divario grazie all'assenza di parti meccaniche, ma la gestione software rimane cruciale. Esistino 2 tecniche di ottimizzazione per il file system:

- ### Tecniche di Ottimizzazione: Caching

  Per minimizzare gli accessi al disco, il Sistema Operativo utilizza la RAM come **Cache**.

  [CACHE: memoria temporanea ad alta velocità che memorizza copie di dati frequentemente utilizzati]

  Il principio fisico sfruttato è quello della *località dei riferimenti*. Esistono due approcci storici:

  - **Buffer Cache:** Opera a basso livello, memorizzando i blocchi grezzi del disco.
  - **Page Cache:** Opera ad alto livello nel VFS (*Virtual File System*), memorizzando pagine di file.

  I sistemi moderni implementano una **Unified Buffer Cache**, che evita la duplicazione dei dati in memoria e ottimizza l'uso della RAM disponibile.
  Per ottimizzare il File System, il Sistema Operativo usa la RAM come Cache, sfruttando il principio di località: invece di andare ogni volta sul disco (che è lento), tiene in memoria i dati usati più spesso.
  A livello di implementazione, il sistema deve gestire due sfide: trovare i dati velocemente e decidere cosa eliminare quando la RAM è piena.

  - Per la ricerca veloce, si usa una Hash Table. Funziona come un indice immediato: il sistema inserisce l'indirizzo del blocco che gli serve e la tabella gli dice istantaneamente se quel blocco è già in RAM o se deve andare a prenderlo sul disco. Questo azzera i tempi di ricerca manuale in memoria.

- Per gestire lo spazio, si usa una lista LRU (Least Recently Used). Possiamo immaginarla come una coda di priorità: i blocchi consultati di recente restano in cima, mentre quelli inutilizzati scivolano verso il fondo. Quando la RAM è piena, il sistema elimina i blocchi che si trovano in fondo alla coda per far posto ai nuovi.

Il punto critico è che non possiamo usare un algoritmo LRU 'puro' per tutto. Se il sistema eliminasse dalla cache un blocco di metadati (come gli i-node, che sono la 'mappa' del disco) prima di averlo salvato fisicamente, e in quel momento ci fosse un crash, l'intero File System diventerebbe illeggibile. Per questo motivo, l'implementazione moderna distingue tra i tipi di dati: i blocchi di dati normali seguono il ritmo della cache, mentre i blocchi critici (i-node) vengono protetti o scritti sul disco con priorità massima. In questo modo si garantisce sempre la coerenza del sistema, evitando che un'interruzione di corrente distrugga la struttura dei file.

• Allocazione dei Blocchi e Read Ahead

Nei dischi magnetici (HDD), il limite principale è rappresentato dal movimento meccanico del braccetto.

- **Allocazione Contigua:** Il sistema cerca di posizionare i blocchi di un file vicini tra loro, preferibilmente nello stesso cilindro.
- **Bitmap vs Lista Libera:** L'uso di una *Bitmap* in RAM permette di individuare istantaneamente il blocco libero più vicino a quello appena scritto, evitando accessi extra al disco per consultare liste di blocchi liberi memorizzate su supporto fisico.
- **Ottimizzazione degli I-node:** Posizionare gli i-node all'inizio del disco causa lunghi spostamenti del braccetto. I sistemi moderni dividono il disco in gruppi, inserendo i-node e dati correlati nello stesso settore per minimizzare i tempi di *seek*.

La creazione e l'eliminazione continua di file genera "buchi" nel disco, portando alla frammentazione.

- **Deframmentazione (Post-process):** Tipica dei sistemi Windows, riorganizza i file per renderli contigui.
- **Preallocazione (In-process):** Utilizzata da File System come *Ext4*, "prenota" preventivamente gruppi di blocchi contigui per i file in espansione, prevenendo la frammentazione alla radice.
- **Nota sugli SSD:** Poiché gli SSD non hanno parti mobili, l'accesso casuale è rapido quanto quello sequenziale. La deframmentazione su questi supporti è inutile e dannosa per la vita utile delle celle Flash.

Per mitigare la lentezza del disco, il Sistema Operativo utilizza la RAM come **Unified Buffer Cache**.

  - **Implementazione:** Si utilizza una *Hash Table* per la ricerca rapida dei blocchi e una lista *LRU (Least Recently Used)* per la gestione dello spazio.
  - **Problema della Coerenza:** Un algoritmo LRU puro non è applicabile ai metadati (i-node). Se un blocco critico venisse rimosso dalla cache prima di essere scritto su disco e avvenisse un crash, il File System risulterebbe illeggibile. Per questo, i sistemi moderni forzano la scrittura dei blocchi critici con priorità elevata.

  Per ottimizzare lo spazio disponibile, si adottano tecniche avanzate di riduzione della ridondanza:

  - **Compressione:** Riduce la dimensione dei file scrivendo dati compressi direttamente sul disco.
  - **Deduplicazione:** Identifica blocchi o file identici (comune in ambienti Cloud) e ne conserva una sola copia fisica, utilizzando dei puntatori logici per gli altri riferimenti. Può essere *inline* (immediata, rallenta la scrittura) o *post-process* (eseguita in background).

Affidabilità del File System

I file non sono al sicuro per sempre. Possono sparire o rovinarsi per vari motivi:

- **Guasti fisici:** Il disco rigido "muore" o si riga.
- **Errori umani:** Quando cancelli per sbaglio la cartella sbagliata (il classico esempio rm * .o che cancella tutto invece di cancellare solo i file specifici).
- **Mancanza di corrente:** Se il PC si spegne mentre sta scrivendo, il file può restare "a metà" e diventare illeggibile.
- **Malware:** Virus che criptano i tuoi dati (Ransomware) chiedendo un riscatto.

Fare il backup non significa solo "copiare i file". Esistono 2 tipologie di Backup:

- Backup Incrementale: Invece di copiare tutto ogni giorno (lungo e pesante), copi tutto una volta a settimana e poi, ogni giorno, copi solo quello che è cambiato dall'ultimo backup.
- Backup Fisico vs Logico:
  - Fisico: Copia il disco blocco per blocco (come fotocopiare ogni singola cella di un foglio a quadretti). Veloce ma rigido.
  - Logico: Copia i file e le cartelle. Più flessibile perché ti permette di recuperare anche un singolo file.

Cosa succede se il PC crasha mentre stai salvando un file? Il File System potrebbe confondersi.
Il **Journaling** è un metodo preventivo. Prima di modificare i dati, il File System scrive un breve record in un'area dedicata chiamata **Journal** (il diario). Il processo segue tre fasi:

1. **Registrazione:** "Ho intenzione di modificare il file X".
2. **Esecuzione:** La modifica viene effettivamente scritta sul disco.
3. **Conferma (Commit):** "Modifica completata con successo".

In caso di crash, il sistema non deve scansionare tutto il disco: legge solo il Journal. Se trova un'operazione registrata ma non confermata, la completa o la annulla, garantendo un ripristino rapidissimo.

Il **VFS** è uno strato di astrazione fondamentale nei sistemi operativi moderni. Il suo scopo è permettere all'utente e alle applicazioni di interagire con diversi tipi di File System (NTFS, ext4, FAT32) come se fossero un'unica struttura coerente.
Il VFS agisce come un "traduttore universale". Quando un processo invia una chiamata di sistema standard (come `open` o `read`), il VFS la intercetta e la smista al driver specifico del File System sottostante. I componenti chiave del VFS sono:

- **Superblock:** Contiene le informazioni generali sul File System montato.
- **V-node:** Una versione astratta dell'I-node. Rappresenta un file in modo universale, indipendentemente dal formato del disco.
- **Vettore di funzioni:** Ogni File System "reale", quando viene montato, fornisce al VFS un elenco di funzioni (come "leggi blocco" o "scrivi file") che il VFS userà per comunicare con lui.

Grazie al VFS, l'utente può copiare un file da una chiavetta USB (FAT32) al disco fisso (NTFS) con un semplice comando, poiché la complessità della traduzione è gestita interamente da questo strato software.

RAID

Il RAID consiste nell'usare più dischi insieme come se fossero uno solo, per due motivi: velocità o sicurezza.

- RAID 0 (Velocità): Spezza il file in due e ne scrive metà su un disco e metà sull'altro. È velocissimo grazie al parallelismo, ma se un disco si rompe, perdi tutto perché non c'è ridondanza.
- RAID 1 (Specchio): Quello che scrivi sul disco A viene copiato identico sul disco B. Se uno si rompe, hai l'altro (migliore affidabilità).

- RAID 2: Usa il codice di Hamming per la correzione degli errori e protegge da letture non corrette provenienti dagli altri dischi.
- RAID 3: I dati vengono suddivisi a livello di byte. Viene aggiunto un bit di parità per ogni parola, archiviato su un disco dedicato.
- RAID 4: Simile al RAID 3, ma la suddivisione dei dati avviene a livello di blocchi. Offre resistenza ai guasti e velocità grazie al parallelismo.
- RAID 5 (Il compromesso): Usa almeno 3 dischi. Distribuisce i dati e la parità su tutti i dischi del sistema. Se un disco si rompe, il sistema usa la matematica (l'operazione XOR) per ricostruire i dati mancanti dagli altri.
- RAID 6: Simile al RAID 5, ma i bit di parità vengono scritti due volte su dischi differenti. Offre un'altissima tolleranza ai guasti grazie alla doppia ridondanza.
- RAID 0+1: Combina la duplicazione dei dati del RAID 1 con la distribuzione a "striping" del RAID 0.